PROJECT REPORT

FOR

WEB PROGRAMMING (CS23531)

AI SHOPPING ASSISTANT – SMART MART INDOOR NAVIGATOR USING A* SEARCH

A PROJECT REPORT

Submitted by

2116241001508 – DEEPAK T



November, 2026

BONAFIDE CERTIFICATE

Certified that this project report "**AI Shopping Assistant – Smart Mart Indoor Navigator using A* Search**" is the bonafide work of "**DEEPAK T - 241001508**" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	1
1	Introduction	2
2	Literature Review	4
3	Proposed System	7
3.1	Architecture Diagram	7
3.2	System Workflow	8
3.3	Problem Formulation for A* Search	9
4	Modules Description	10
4.1	Store Map and Product Data Module	10
4.2	A* Search Module	11
4.3	Route Planning Module	12
4.4	Instruction Generation Module	12
4.5	User Interface and Map Rendering Module	13
5	Implementation And Results	15
5.1	Experimental Setup	15
5.2	Results	15
6	Conclusion And Future Work	20
7	References	21

ABSTRACT

Finding products inside a large supermarket is time-consuming, and customers often walk the same aisles repeatedly because most stores offer only static signboards. GPS does not work reliably indoors, and commercial indoor-positioning systems need extra hardware. This project presents the AI Shopping Assistant, a browser-based indoor navigation system for a Tamil Nadu supermarket that uses the A* search algorithm to plan walking routes. The store is modelled as a 39 × 30 grid graph containing eleven product sections, an entrance and four billing counters, and a catalogue of 90 products is mapped onto shelf positions. The customer selects products, and the system orders the stops with a nearest-neighbour rule, computes the shortest path for every leg using A* with the Manhattan Distance heuristic, and selects a billing counter. The same path is used to draw the route on a live floor map and to generate turn-by-turn text directions, so the two always agree. The system is written in plain HTML, CSS and JavaScript and runs without a server, GPS or map API. On five test shopping lists of one to ten products the routes ranged from 31 to 115 steps, and A* path lengths matched breadth-first search on all 120 checked start–destination pairs.

CHAPTER 1
INTRODUCTION

1.1 Background

Supermarkets stock thousands of items across many aisles, and a customer who is unfamiliar with the layout usually has to walk around the store looking for each product. Overhead signboards show only the general category of an aisle, and store staff are not always available to give directions. The result is wasted time, repeated walking over the same aisles, and an unpleasant shopping experience, especially when the shopping list contains items from many different sections.

Route finding is a classic problem in Artificial Intelligence. When the environment can be represented as a graph, a search algorithm can find a path from a start state to a goal state automatically. This project applies that idea to a supermarket: the store floor is represented as a graph, each product is a goal location, and an informed search algorithm, A* (A-Star), finds the shortest walking route between locations.

1.2 Problem Statement

Customers spend too much time searching for products in large supermarkets because there is no efficient navigation system inside the store. Outdoor navigation tools such as GPS-based maps cannot be used inside a building, and hardware-based indoor positioning needs beacons or sensors that most stores do not have. A software-only solution that can guide a customer to every item on a shopping list, and then to a billing counter, is therefore needed.

1.3 Objectives

- To represent a supermarket floor plan as a walkable graph (grid) with shelves treated as obstacles.
- To implement the A* search algorithm with the Manhattan Distance heuristic and to compute the shortest path between any two locations in the store.

- To let a customer search a product catalogue, build a shopping list and obtain one combined route from the entrance through every product to a billing counter.
- To present the route in two forms generated from a single source: a live route drawn on a floor map and dynamic turn-by-turn text instructions.
- To build the system without any external map service, GPS or server, so that it runs by simply opening a web page.

1.4 Scope and Limitations

The system models a single-floor Tamil Nadu supermarket ("Smart Mart") with eleven product sections, a trolley bay, one entrance and four billing points (three billing counters and a self-checkout). The product catalogue contains 90 typical grocery, household and personal-care items. The customer's starting point is always the entrance; the system does not track the customer's live position, queue lengths or stock levels. The store layout and catalogue are stored as static data inside the program and can be edited by changing the data files.

CHAPTER 2
LITERATURE REVIEW

2.1 Uninformed Graph Search

Uninformed search strategies such as breadth-first search (BFS) and depth-first search (DFS) explore a graph without any knowledge of where the goal lies (Russell and Norvig, 2021). BFS expands nodes level by level and returns a shortest path when every move has the same cost, but it spreads in all directions and may examine a large part of the graph before reaching the goal. DFS follows one branch as deep as possible; it uses little memory but does not guarantee the shortest path and may return a long, winding route. For route guidance inside a store, returning the shortest path is essential, so DFS is unsuitable.

2.2 Shortest-Path Algorithms

Dijkstra (1959) described an algorithm that finds the cheapest path from a source node to all other nodes of a weighted graph by always expanding the node with the lowest cost so far. It is guaranteed to be optimal, but because it uses only the cost already travelled, it also expands nodes that lie away from the destination. Cormen et al. (2009) discuss it together with other shortest-path algorithms as a standard tool of algorithm design.

2.3 A* Search and Heuristics

Hart, Nilsson and Raphael (1968) introduced A*, which extends Dijkstra's idea with a heuristic estimate of the remaining cost. A* expands the node with the smallest value of $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start to n and $h(n)$ is the estimated cost from n to the goal. If $h(n)$ never overestimates the true remaining cost (an admissible heuristic), A* is guaranteed to return an optimal path, and it normally examines far fewer nodes than uninformed search because the heuristic pulls the search towards the goal (Russell and Norvig, 2021). Pearl (1984) gives a detailed treatment of heuristic design. On a grid where movement is restricted to up, down, left and right, the Manhattan Distance $|x1 - x2| + |y1 -$

y2| is a natural admissible heuristic, because no path can be shorter than the number of horizontal and vertical steps between two cells.

2.4 Indoor Positioning and Navigation

Outdoor navigation relies on satellite positioning, which is unreliable inside buildings. Research on in-building location systems, such as the RADAR system of Bahl and Padmanabhan (2000), estimates a user's position from radio-signal strength measured by a network of base stations. Such approaches need installed infrastructure and calibration. In contrast, the present project assumes a known store map and a fixed starting point, so that route planning can be done entirely in software, and it concentrates on the AI search problem of computing the best route.

2.5 Visiting Several Locations

A shopping list with many items is a multi-stop routing problem, closely related to the travelling salesman problem. Finding the exact best visiting order is computationally expensive for long lists, so simple heuristics are used in practice. The nearest-neighbour rule always moves to the closest unvisited location and is analysed, along with other heuristics, by Rosenkrantz, Stearns and Lewis (1977). It is fast and easy to explain but does not always give the globally shortest tour; this project uses it only to decide the visiting order, while A* computes the real path of every leg.

Table 2.1 Comparison of search algorithms for store navigation

Algorithm	Uses path cost	Uses heuristic	Shortest path guaranteed	Remark
DFS	No	No	No	May return long, winding routes
BFS	Step count	No	Yes (equal step cost)	Explores in all directions
Dijkstra	Yes	No	Yes	Expands nodes away from the goal
A* (used)	Yes	Yes (Manhattan)	Yes (admissible h)	Guided towards the goal; fewer nodes

CHAPTER 3
PROPOSED SYSTEM

The proposed AI Shopping Assistant is a web application that guides a customer through the store in five steps: (1) select the products to buy, (2) find the location of each product, (3) calculate an efficient route with A* search, (4) follow map-based and text-based guidance, and (5) reach the billing counter. All calculations are done in the customer's browser; no GPS, map API or server is involved, and the route is never hard-coded – it is computed afresh for every shopping list.

3.1 ARCHITECTURE DIAGRAM

The system is organised into four layers, shown in Figure 3.1. The presentation layer (HTML, CSS, guidance panel and map renderer) interacts with the customer. The controller, app.js, keeps the application state and handles every user event; it never calculates a route itself. The routing-logic layer converts a shopping list into a complete route and instructions, and it calls the A* search engine for every path it needs. The data layer holds the store map and product catalogue used by the search engine.

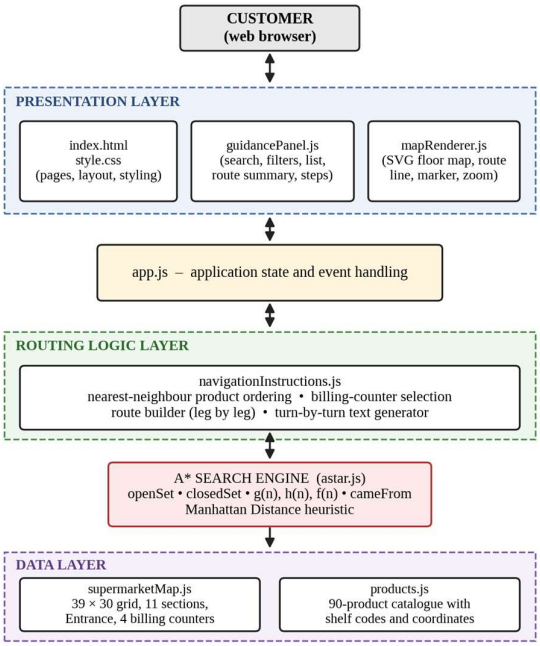


Figure 3.1 Architecture of the AI Shopping Assistant

3.2 SYSTEM WORKFLOW

Figure 3.2 shows the workflow of one shopping trip. The customer builds a shopping list and selects a billing counter. When “Find Best Route” is pressed, the products are ordered with the nearest-neighbour rule starting from the entrance, and the billing counter is fixed (either the one chosen by the customer or, in “Nearest” mode, the counter with the smallest real A* distance from the last product). A* is then run once for every consecutive pair of stops, and the leg paths are joined into one route. This single list of coordinates is used twice: the map renderer converts it into the route line, and the instruction generator converts it into text, so the map and the text can never disagree.

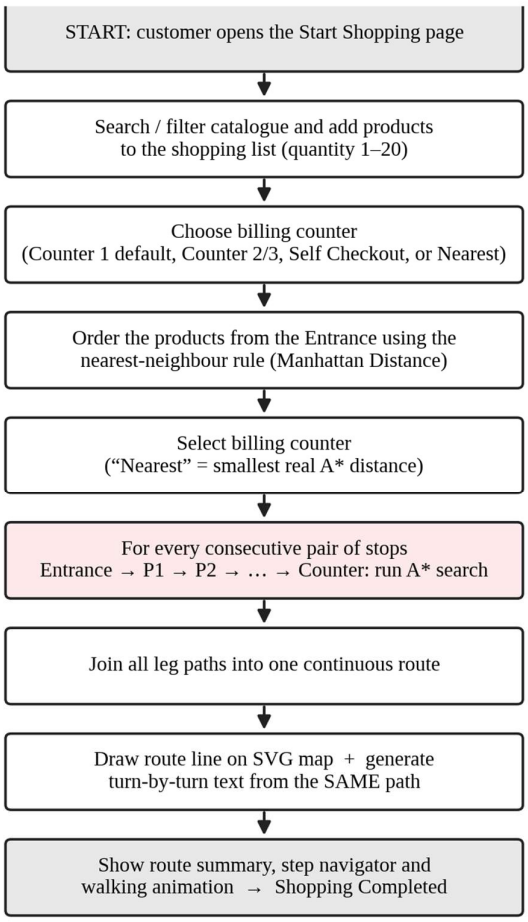


Figure 3.2 Workflow of route planning

3.3 PROBLEM FORMULATION FOR A* SEARCH

Each leg of the journey is posed as a standard search problem, summarised in Table 3.1. The store is a two-dimensional array in which 0 marks walkable floor and 1 marks a shelf or wall. A node is a cell (x, y); from a cell the agent may move one step up, down, left or right into any neighbouring cell whose value is 0, and every move costs 1.

Table 3.1 A* formulation of the in-store navigation problem

Component	Definition in this project
State	A walkable grid cell (x, y)
Initial state	Entrance, or the previous stop of the route
Goal state	Shelf position of the next product, or the chosen billing counter
Actions	Move Up, Down, Left or Right into a cell whose value is 0 (no diagonals)
Step cost	1 per move
g(n)	Number of steps taken from the start of the leg to n
h(n)	Manhattan Distance from n to the goal: $\|x - x_goal\| + \|y - y_goal\|$
f(n)	g(n) + h(n); the open node with the smallest f(n) is expanded next

Because movement is limited to four directions, the Manhattan Distance can never be larger than the true remaining distance, so h(n) is admissible and the path returned by A* is the shortest possible for that leg.

CHAPTER 4
MODULES DESCRIPTION

The implementation is divided into five modules. Each module is a separate JavaScript file (or group of files) with one clear responsibility, and only app.js reads user-interface events.

4.1 STORE MAP AND PRODUCT DATA MODULE

This module (data/supermarketMap.js and data/products.js) defines the store. The floor is built from layout constants: every section is 6 units wide and 5 units deep, aisles between sections are 3 units wide, the outer margin is 3 units, the entrance lobby is 4 units deep and the billing lobby is 5 units deep. These give a grid of 39 × 30 cells. Twelve rectangular zones are arranged in a 4 × 3 table: eleven product sections and one decorative Trolley Bay. Aisles and section floors are carved as walkable cells, and a shelf rack of obstacle cells is placed through the middle of every product section, so that A* must route around it (Figure 4.1).

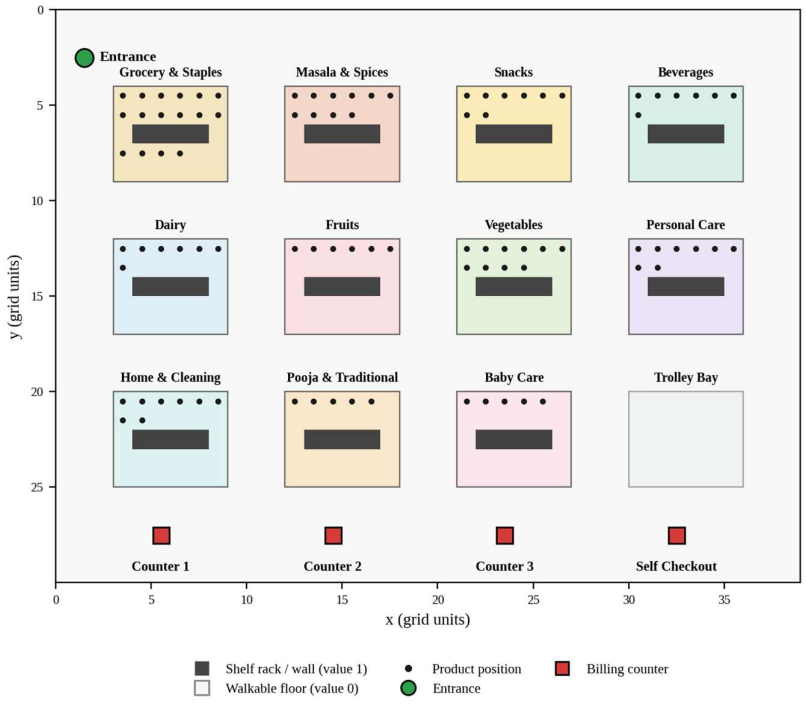


Figure 4.1 Store layout used by the A* engine (39 × 30 grid)

The Entrance is fixed at cell (1, 2). The four billing points – Billing Counter 1, Billing Counter 2, Billing Counter 3 and Self Checkout – are fixed at y = 27 and x = 5, 14, 23 and 32. The product catalogue holds 90 items distributed over the eleven sections (Table 4.1). Each product record has an id, name, category, section, shelf code (for example GR-02), grid coordinates, icon, price and unit. Shelf codes and coordinates are generated automatically from the open slots of the section, so a new product is added by writing one line in the catalogue.

Table 4.1 Products per store section

Section	Products	Section	Products
Grocery & Staples	16	Personal Care	8
Masala & Spices	10	Home & Cleaning	8
Snacks	8	Pooja & Traditional	5
Beverages	7	Baby Care	5
Dairy	7	Vegetables	10
Fruits	6	**Total**	**90**

4.2 A* SEARCH MODULE

The file algorithms/astar.js contains the search engine. It works only on grids and coordinates and never touches the user interface, so it can be tested on its own. The function aStarSearch(grid, width, height, start, goal) keeps the structures named in the textbook description of A*: an open set of discovered but unevaluated nodes, a closed set of fully evaluated nodes, and the maps gScore, hScore, fScore and cameFrom. The procedure is:

1. Put the start cell in the open set with g = 0, h = Manhattan Distance to the goal and f = g + h.
2. Select the open-set node with the smallest f(n). If it is the goal, rebuild the path by following cameFrom backwards and return it.
3. Otherwise move the node from the open set to the closed set and examine its four neighbours (getWalkableNeighbors), accepting only cells that lie inside the grid and have value 0.

4. For each neighbour not in the closed set, compute the tentative g = g(current) + 1. If the neighbour is new, or this g is smaller than its recorded g, store the parent, g, h and f of the neighbour and add it to the open set.
5. Repeat from step 2. If the open set becomes empty, no path exists and null is returned.

The open node with the smallest f is found by a linear scan. For a grid of 1,170 cells this is fast enough; a larger map would use a binary-heap priority queue.

4.3 ROUTE PLANNING MODULE

This module is part of utils/navigationInstructions.js. It turns a shopping list into a route in the following way. First, orderProductsByNearestNeighbor starts at the Entrance and repeatedly chooses the closest remaining product by Manhattan Distance; this decides only the visiting order. Second, pickBillingCounter returns the counter chosen by the customer (Billing Counter 1 is the default) or, in "Nearest" mode, runs A* from the last product to all four counters and picks the one with the smallest real walking distance. Third, buildShoppingRoute calls A* separately for every pair of consecutive stops in the sequence Entrance → P1 → P2 → … → Billing Counter, and adds up the number of steps. If any leg has no path, an error message is shown to the customer. The estimated shopping time in minutes is calculated as ceil((steps × 1.2 + items × 12) / 60), which assumes 1.2 seconds per step and 12 seconds to pick each item.

4.4 INSTRUCTION GENERATION MODULE

The legs of the route are joined into one continuous list of coordinates, and the position of every product and of the billing counter in this list is recorded. generateTextInstructions then walks along the list, groups consecutive moves in the same direction into one instruction, and compares each new direction with the previous one to decide whether the customer must turn left, turn right, continue straight or turn around. Every instruction is labelled with the section the segment is heading towards (looking ahead if the path is still in an aisle), and a separate

line is added whenever a product is reached ("Collect … – Shelf …") or the billing counter is reached. Nothing is hard-coded; the text is produced entirely from the coordinates returned by A*. Table 4.2 shows the instructions generated for a list of Ponni Rice and Milk.

Table 4.2 Instructions generated for "Ponni Rice + Milk" (Billing Counter 1)

Step	Instruction
1	Starting Point: Entrance
2	Walk straight toward Grocery & Staples
3	Turn left toward Grocery & Staples
4	Collect Ponni Rice – Shelf GR-02
5	Turn right toward Grocery & Staples
6	Turn right toward Grocery & Staples
7	Turn left toward Dairy
8	Collect Milk – Shelf DA-01
9	Continue straight toward the Billing area
10	Turn left toward the Billing area
11	Proceed to Billing Counter 1.

4.5 USER INTERFACE AND MAP RENDERING MODULE

The interface consists of index.html, css/style.css, components/guidancePanel.js, components/mapRenderer.js and app.js. The application has three pages: Home, Start Shopping and About & Viva Prep. On the Start Shopping page the customer can search the catalogue by name, filter by category, open a product pop-up, add or remove items, change quantities between 1 and 20 and choose a billing counter. mapRenderer.js draws the floor plan as scalable vector graphics (SVG): rounded coloured sections, product markers, the entrance and billing points, and – after a route is calculated – a thick route line with direction arrows and an animated marker. The map supports zoom in, zoom out and re-centre. guidancePanel.js displays the turn-by-turn list, a step indicator with Previous and Next buttons, a route summary (number of products, start, destination, algorithm, heuristic, total

distance and estimated time) and a “Start Walking Animation” option. When the customer finishes, a shopping summary lists the collected items, the total distance and the billing counter. Changing the shopping list while a route is displayed clears the route so that an outdated route is never shown.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The system is implemented with standard web technologies only (Table 5.1). There is no framework, build tool, database or server; the application is started by opening index.html in a web browser, and it works without an internet connection (Google Fonts is used for headings and falls back to system fonts when offline). The A* engine and routing code were also loaded in Node.js to run the test shopping lists and the correctness check described below, and the screenshots in this chapter were captured from the running application in a Chromium-based browser.

Table 5.1 Implementation environment

Item	Details
Programming languages	HTML5, CSS3, JavaScript (ES6)
Map drawing	Scalable Vector Graphics (SVG) generated by JavaScript
AI technique	A* search, Manhattan Distance heuristic; nearest-neighbour ordering
Store model	39 × 30 grid, 11 product sections, 1 entrance, 4 billing points
Data	90-product catalogue held in a JavaScript file (no database)
Run-time	Any modern web browser; no server, GPS or map API
Testing tools	Node.js (algorithm tests), Chromium browser (interface)

5.2 RESULTS

5.2.1 Application output

Figure 5.1 shows the Home page, which explains the five steps of the system. Figure 5.2 shows the Start Shopping page with four products added to the shopping list. After “Find Best Route” is pressed, the map shows the A* route from the entrance through the product sections to Billing Counter 1, and the Navigation Guidance panel lists the generated directions together with the route summary (Figure 5.3). Figure 5.4 shows the route for a longer list of ten products, and Figure 5.5 shows the shopping summary displayed on completion.

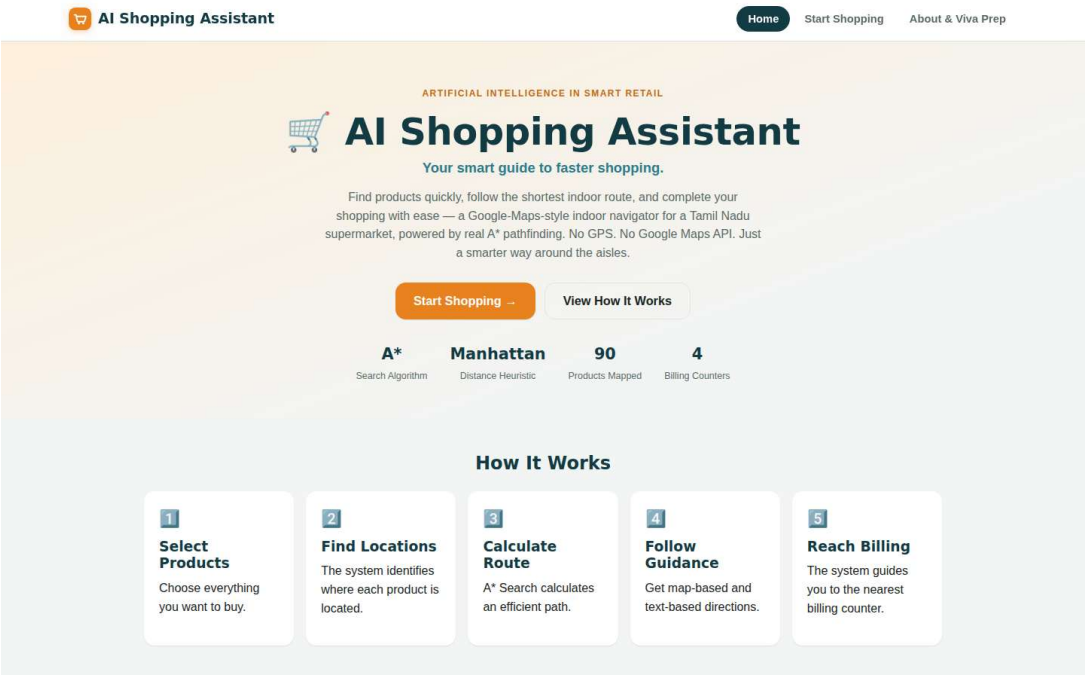


Figure 5.1 Home page

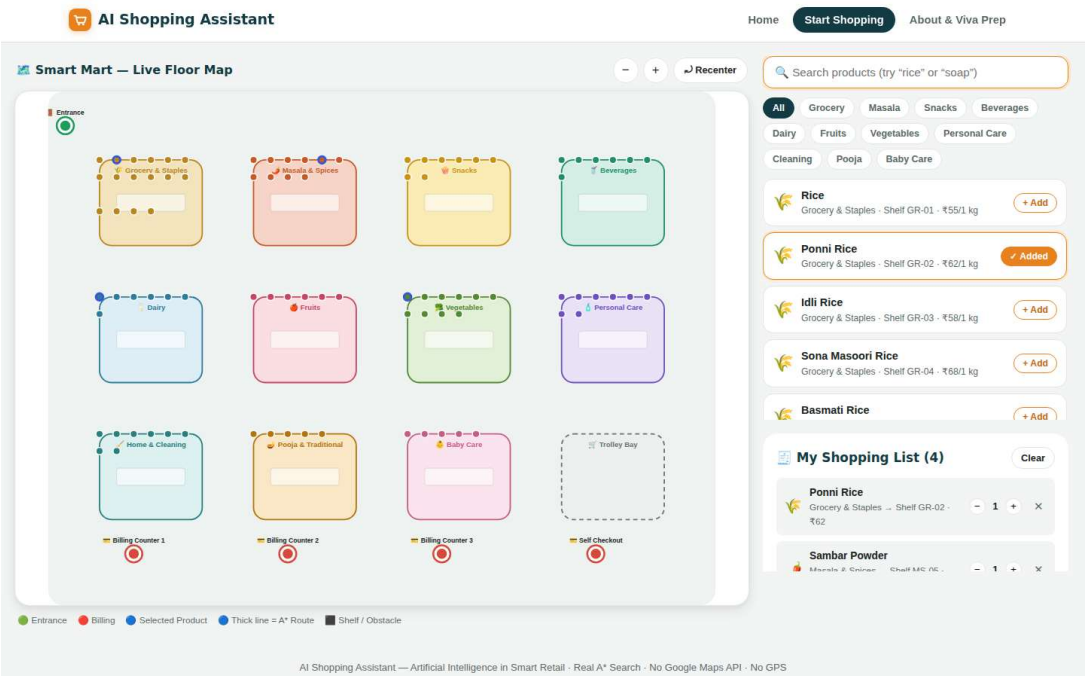


Figure 5.2 Product search and shopping list

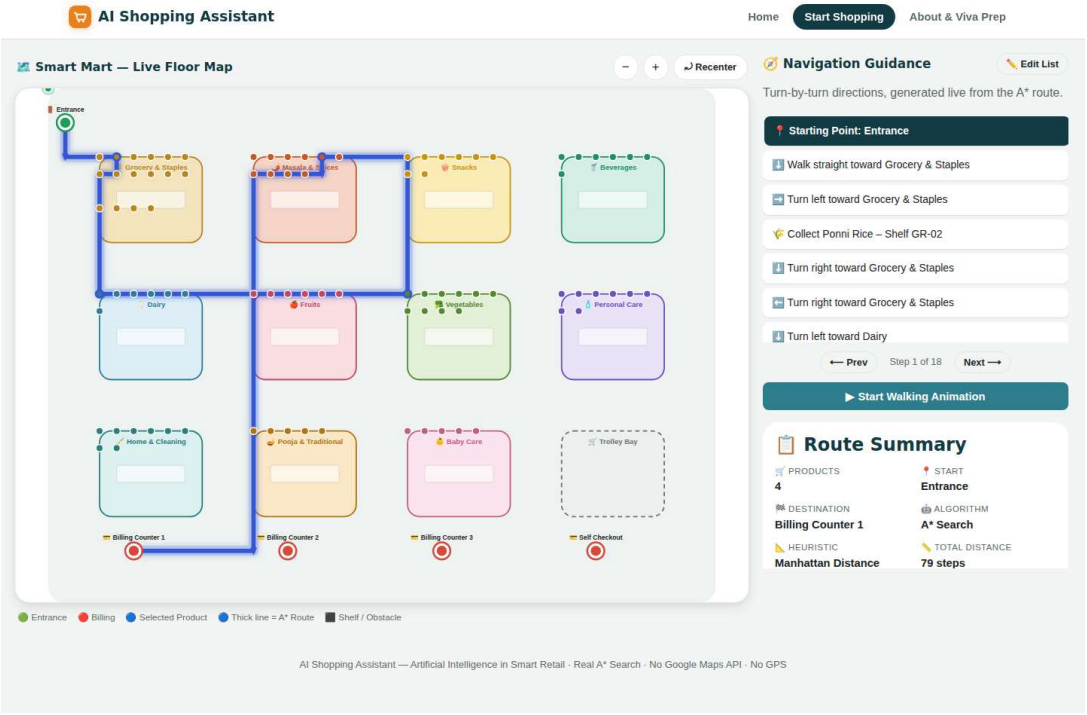


Figure 5.3 Route and text guidance for Ponni Rice, Sambar Powder, Milk and Tomato

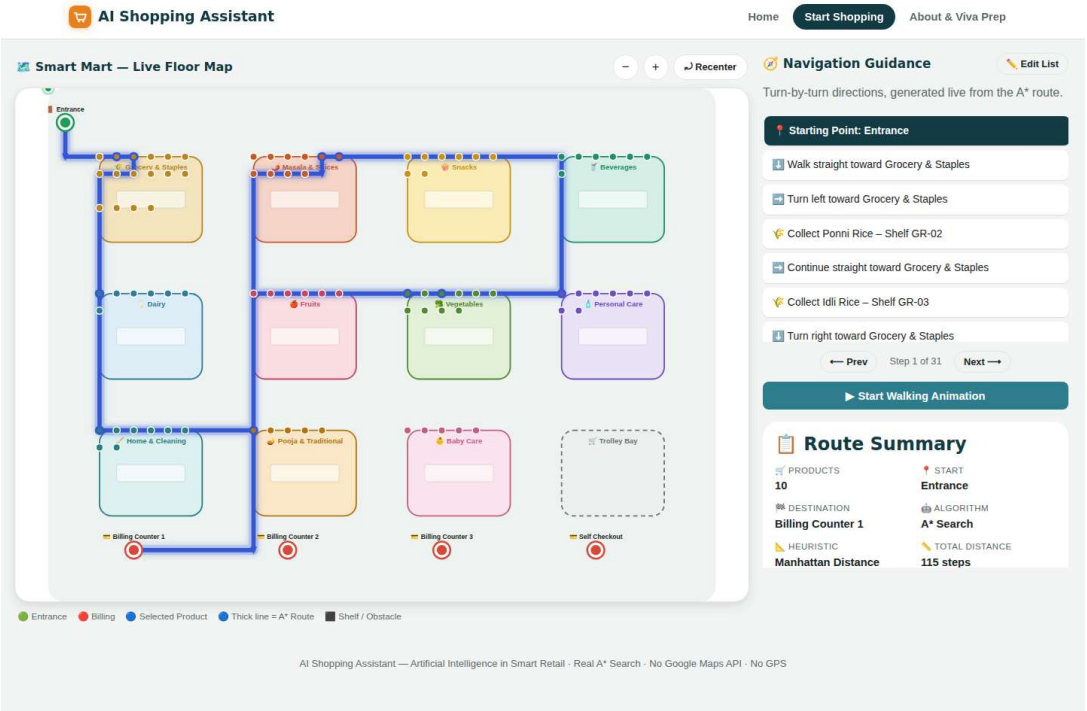


Figure 5.4 Route for a shopping list of ten products

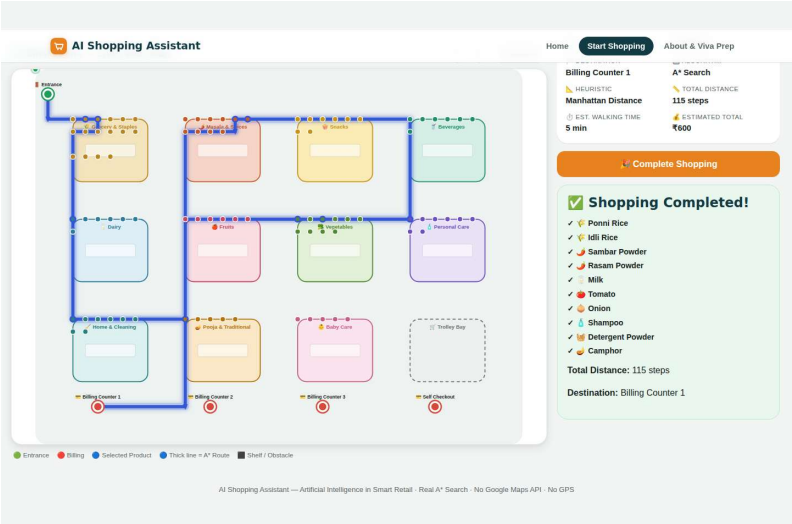


Figure 5.5 Shopping summary after completing the route

5.2.2 Test cases

Five shopping lists of increasing size were tested, each with the default Billing Counter 1 and with the "Nearest" option. Distances are counted in grid steps. The results are given in Table 5.2 and Figure 5.6.

Table 5.2 Route results for the test shopping lists

Test	Shopping list	Steps with Counter 1	Nearest counter	Steps with nearest	Saving	Time (min)	Text lines
1	Ponni Rice	31	Counter 1	31	0%	1	9
2	Ponni Rice, Milk	31	Counter 1	31	0%	2	11
3	Ponni Rice, Sambar Powder, Milk, Tomato	79	Counter 2	72	8.9%	3	18
4	Rice, Murukku, Banana Chips, Shampoo, Detergent Powder	111	Self Checkout	88	20.7%	4 / 3	19
5	Ponni Rice, Idli Rice, Sambar Powder, Rasam Powder, Milk, Tomato, Onion, Shampoo, Detergent Powder, Camphor	115	Counter 2	108	6.1%	5	31

Time is the estimate for Counter 1 (and for the nearest counter where it differs). Text lines is the number of instruction lines generated.

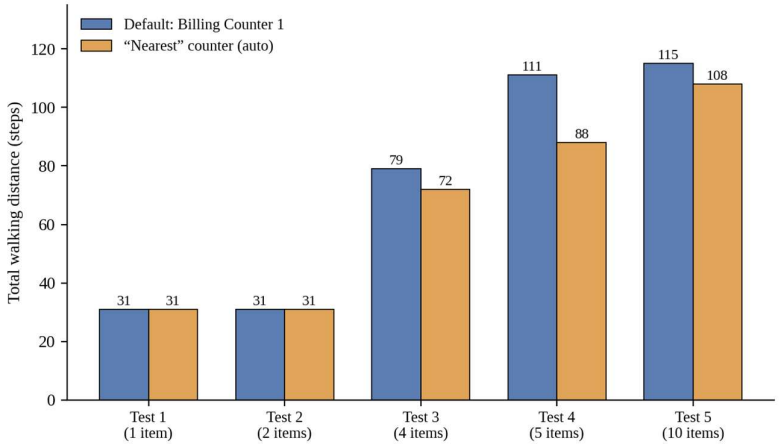


Figure 5.6 Total walking distance for the five test cases

5.2.3 Correctness check

To confirm that the implementation returns shortest paths, the A* path length was compared with the length found by breadth-first search on the same grid for 120 pairs, formed by taking every third product of the catalogue and each of the four billing points. The lengths were equal in all 120 pairs. In addition, A* found a path from the Entrance to all 90 products, which shows that no product is sealed off by the shelf racks.

5.2.4 Discussion

The results show that the route length grows with the number of sections that have to be visited, from 31 steps for one or two nearby items to 115 steps for a ten-item list. Test 2 has the same distance as Test 1 because Milk lies on the way from Ponni Rice to Billing Counter 1. Letting the system choose the billing counter reduced the walking distance by 8.9%, 20.7% and 6.1% in Tests 3, 4 and 5, because the last product was closer to another counter than to Counter 1. For short lists whose last stop was near Counter 1, no saving was possible. The nearest-neighbour rule only decides the visiting order, so the complete route is not guaranteed to be the shortest possible tour; the individual legs, however, are optimal.

CHAPTER 6
CONCLUSION AND FUTURE WORK

This project developed an AI-based indoor shopping assistant that guides a customer through a supermarket using the A* search algorithm. The store is modelled as a grid graph, A* with the Manhattan Distance heuristic finds the shortest path for every leg of the journey, and the resulting route is shown both as a live line on a floor map and as dynamically generated turn-by-turn text. The system runs completely in the browser without GPS, a map service or a server. Tests on shopping lists of one to ten products produced valid routes of 31 to 115 steps, A* path lengths agreed with breadth-first search in all 120 checked cases, and choosing the nearest billing counter reduced the walking distance by up to 20.7%.

The present system has some limitations: it uses a fixed single-floor layout, assumes that every journey starts at the entrance, does not track the customer's live position, and decides the visiting order with a greedy nearest-neighbour rule. The following enhancements are possible:

- Replace the linear scan for the lowest f(n) by a binary-heap priority queue for larger stores.
- Find a better visiting order with an improved tour heuristic (such as 2-opt) or an exact method for short lists.
- Load the store layout and product data from a database so that stock changes and new layouts need no code change.
- Add live positioning (for example QR codes at aisle ends or Bluetooth beacons) so that the route can be recalculated if the customer leaves it.
- Support several floors, weighted aisle costs for crowded areas, voice guidance and accessibility options.
- Port the application to a React/TypeScript or mobile-app version and evaluate it with real shoppers.

CHAPTER 7
REFERENCES

1. Bahl, P. and Padmanabhan, V.N. (2000) 'RADAR: An In-Building RF-Based User Location and Tracking System', Proceedings of IEEE INFOCOM 2000, Vol. 2, pp. 775–784.
2. Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) Introduction to Algorithms, 3rd Edition, MIT Press, Cambridge, MA.
3. Dijkstra, E.W. (1959) 'A Note on Two Problems in Connexion with Graphs', Numerische Mathematik, Vol. 1, No. 1, pp. 269–271.
4. Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', IEEE Transactions on Systems Science and Cybernetics, Vol. 4, No. 2, pp. 100–107.
5. Pearl, J. (1984) Heuristics: Intelligent Search Strategies for Computer Problem Solving, Addison-Wesley, Reading, MA.
6. Rosenkrantz, D.J., Stearns, R.E. and Lewis, P.M. (1977) 'An Analysis of Several Heuristics for the Traveling Salesman Problem', SIAM Journal on Computing, Vol. 6, No. 3, pp. 563–581.
7. Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th Edition, Pearson, Hoboken, NJ.